

Week 1 — Dashboard Scaffolding (Day 1–5)

Day 1 (Today)

- Init React project (Vite) in the repo, folder structure (src/components, src/pages, src/services).
- Install dependencies: recharts, axios (or fetch wrapper), react-router-dom.
- Basic README stub.
- **Commit:** chore: initialize React project structure and dependencies

Day 2

- Build routing: Home page → Shipment Dashboard page.
- Build layout shell — Header component, main content area.
- **Commit:** feat: add routing and base layout shell

Day 3

- Build the search bar component (input + submit button, controlled state).
- Create src/services/shipmentService.js — this is your key trick. Add stub functions: getShipmentById(id), getShipmentEvents(id), getShipmentState(id, timestamp) — all returning **mock JSON** for now.
- **Commit:** feat: add search bar component and mock shipment service layer

Day 4

- Wire search bar to getShipmentById() mock function.
- Display result (basic card showing shipment ID + mock current state fields).
- **Commit:** feat: connect search bar to mock shipment data

Day 5

- Add loading state (spinner while "fetching"), error state (invalid/unknown ID), empty state.
- Basic CSS/styling pass on Week 1 components.
- **Commit:** polish: add loading, error, and empty states to dashboard

Week 2 — Timeline UI (Day 6–10)

Day 6

- Build vertical Timeline component — static structure, rendering a mock array of events (type, timestamp, payload).
- **Commit:** feat: add timeline component with mock event data

Day 7

- Style the timeline: event type badges/icons, formatted timestamps, payload summary text.
- **Commit:** style: polish timeline event cards and layout

Day 8

- Connect Timeline to `getShipmentEvents(id)` from your service layer (still mock, but now properly consumed as "the" data source instead of hardcoded).
- **Commit:** feat: wire timeline component to shipment service layer

Day 9

- Check in with P3 — if `GET /shipment/:id/events` is live, swap the mock inside `getShipmentEvents()` for a real `axios/fetch` call. If not ready yet, keep mock but leave a clear `// TODO: swap to real endpoint` comment.
- **Commit:** feat: integrate real events endpoint (or chore: prep events endpoint for integration if still waiting)

Day 10

- Edge cases: shipment with zero events, shipment with many events (scroll behavior), invalid shipment ID handling in timeline.
- **Commit:** fix: handle timeline edge cases (empty, invalid, long lists)

(Mid-project review lands around here — timeline + reconstruction should be demoable.)

Week 3 — State Scrubbing (Day 11–15)

Day 11

- Build the slider UI component — static, no logic yet, just the visual control with a date/time label.
- **Commit:** feat: add state-scrubbing slider UI (static)

Day 12

- Wire slider's `onChange` to `getShipmentState(id, timestamp)` mock function; display returned mock "historical state" below the slider.
- **Commit:** feat: connect slider to mock historical state

Day 13

- Swap mock for real `GET /shipment/:id/state?at=timestamp` endpoint once P1 ships it.
- **Commit:** feat: integrate real state-at-timestamp endpoint

Day 14

- UX: clearly distinguish "Live" vs "Viewing: [date]" mode, disable slider while fetching, loading indicator during scrub.
- **Commit:** feat: add live vs historical state indicator and loading UX

Day 15

- Edge cases: timestamp before shipment creation, timestamp in the future, rapid slider dragging (debounce the API calls).
 - **Commit:** fix: handle scrubber edge cases and debounce requests
-

Week 4 — Charts, Conflict Handling & Polish (Day 16–20)

Day 16

- Integrate Recharts — basic line chart with mock sensor/temperature data.
- **Commit:** feat: add Recharts sensor visualization (mock data)

Day 17

- Swap mock sensor data for real data once P3 ships TEMPERATURE_SPIKE-type events.
- **Commit:** feat: integrate real sensor event data into chart

Day 18

- Overlay chart with event timeline markers (align spike events visually with chart points).
- **Commit:** feat: overlay event markers on sensor chart

Day 19

- Handle P1's 409 conflict responses gracefully in UI (toast/error message instead of crash), general empty-state pass across the whole app.
- **Commit:** feat: add graceful 409 conflict handling and final empty states

Day 20

- Final polish pass: consistent styling, README update with setup instructions + architecture summary, demo script/notes.
- **Commit:** docs: finalize README and polish UI for final review